

Ejercicio 8: Ejercicios sobre metodologías de desarrollo en cascada

Integrantes: Danilo Cerasa, Lara Pollastrini, Ignacio Abranchuk, Santiago Passerini e Ignacio Criscenti

1. Análisis de requerimientos:

Ejercicio 1: Un cliente te solicita una aplicación web para gestionar su inventario. Define los requisitos funcionales y no funcionales del sistema.

Requisitos funcionales:

1. **Registro de productos:** Los usuarios deben poder ingresar nuevos productos al sistema, proporcionando información como nombre, descripción, categoría, cantidad inicial.
2. **Búsqueda Y Filtrado de productos:** Los usuarios deben poder buscar y filtrar productos por nombre o categoría para localizar rápidamente elementos específicos en el inventario.
3. **Actualización de inventarios:** El sistema debe permitir a los usuarios actualizar las cantidades disponibles de productos en el inventario, ya sea agregando, eliminando o ajustando existencias.
4. **Gestión de proveedores:** Debe ser posible registrar y gestionar información de proveedores, asociando productos a proveedores específicos y manteniendo datos de contacto relevantes.
5. **Generación de Reportes:** Los usuarios deberían poder generar informes sobre el estado actual del inventario, incluyendo detalles como existencias disponibles, los tres productos con mayor y menor stock.

Requisitos no funcionales:

1. Usabilidad:

- a. La interfaz de usuario debe ser intuitiva y fácil de usar, considerando usuarios no técnicos que gestionarán el inventario.

2. Rendimiento:

- a. El sistema debe ser eficiente y responder rápidamente, incluso con grandes volúmenes de datos.

3. Escalabilidad:

- a. Debe ser capaz de manejar un crecimiento futuro en términos de cantidad de productos y usuarios concurrentes.

4. Seguridad:

- a. Deben implementarse medidas de seguridad robustas para proteger la integridad y confidencialidad de los datos del inventario.

5. Disponibilidad:

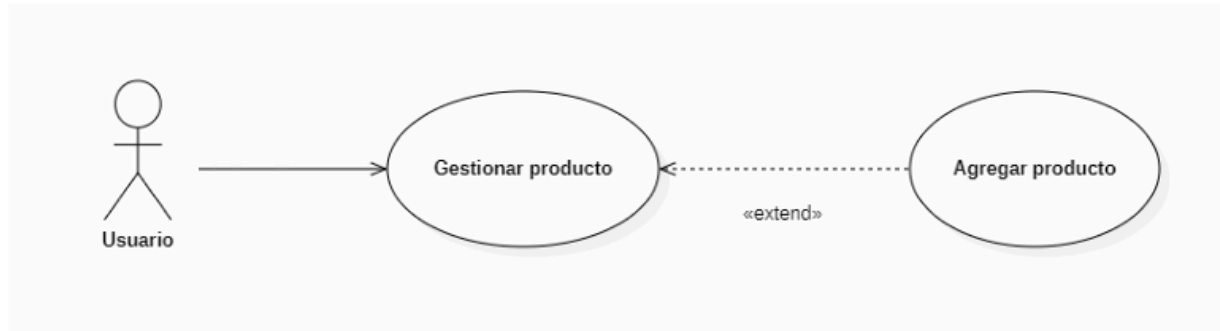
- a. El sistema debe estar disponible en todo momento, minimizando tiempos de inactividad no planificados.

6. Compatibilidad:

- a. Debe ser compatible con diferentes navegadores web y dispositivos para garantizar una experiencia consistente para los usuarios.

Ejercicio 2: Redacta un caso de uso para la funcionalidad de "Agregar un nuevo producto" en la aplicación web del ejercicio 1.

Diagrama



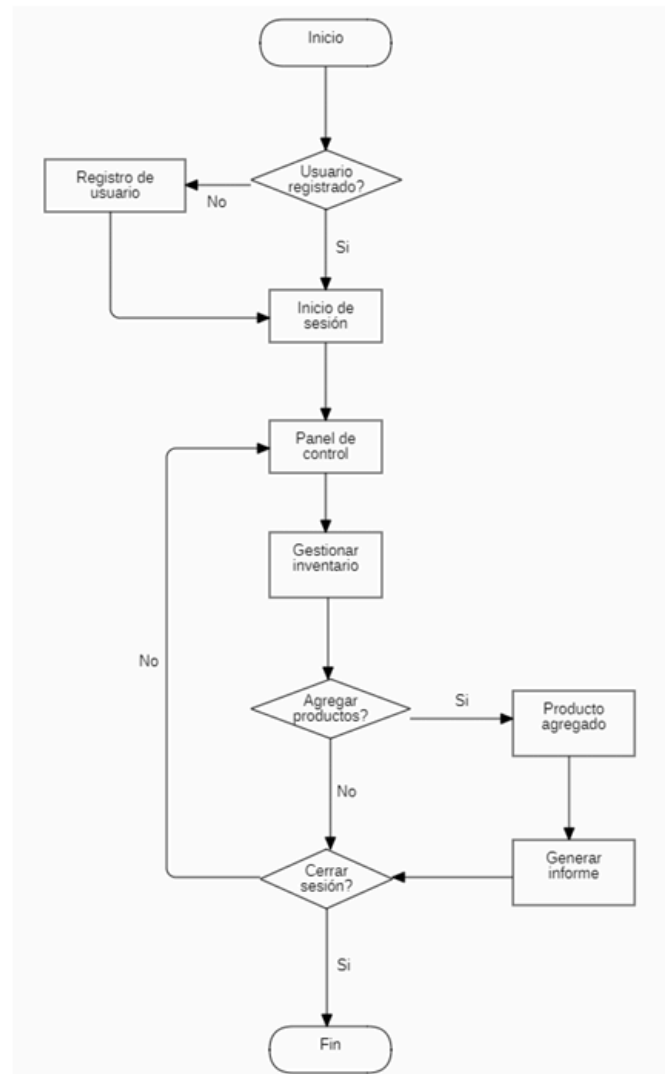
Especificación

	Caso de Uso(CUAN)	Fecha: 07/05/26
Código	CUAN 001	
Nombre	Agregar producto	
Referencias		
Autor	Danilo Cerasa, Ignacio Abranchuk, Lara Pollastrini, Santiago Passerini Ignacio Criscenti	
Revisor	Alejandro Sartorio	
Versión	1.0	
Estado		
Descripción	El usuario podrá agregar un nuevo producto.	
Actores	Usuario	
Pre-condición	El usuario debe tener los permisos correspondientes para poder agregar un nuevo producto.	
CU Extensión	-	
Puntos de Extensión		
Pos-Condición	El nuevo producto se agrega correctamente al inventario del sistema.	
Curso Básico		

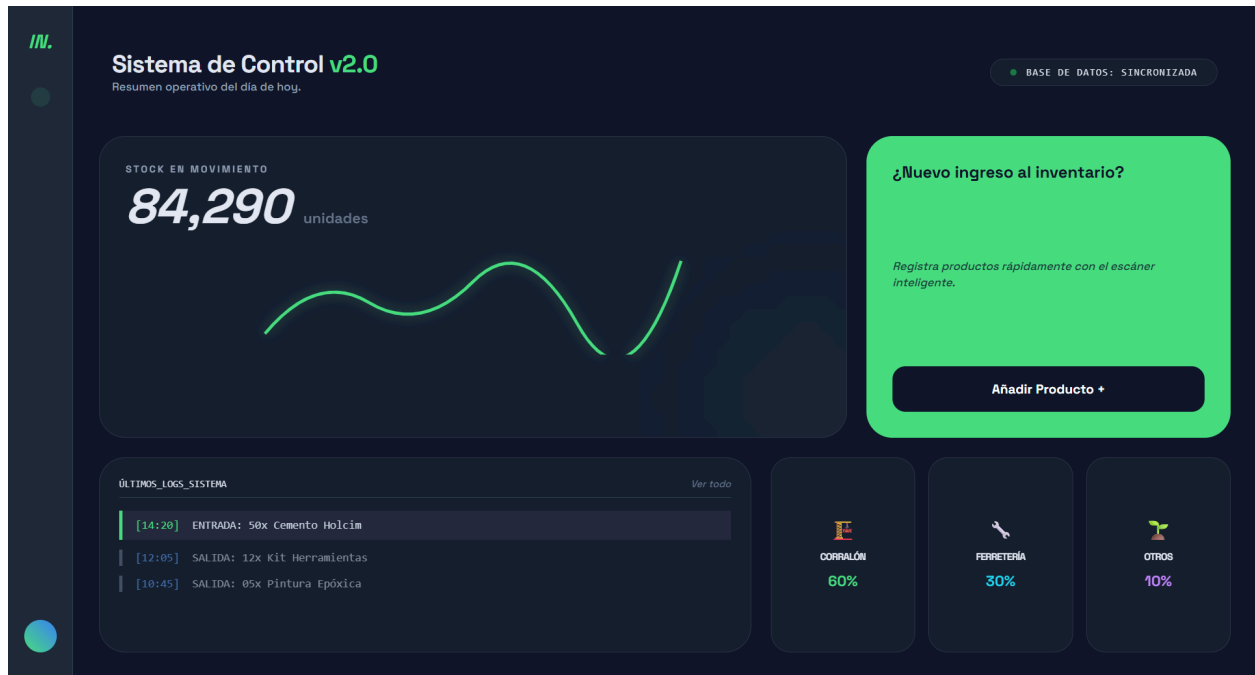
	Caso de Uso(CUD)	Fecha: 07/05/26
Código	CUD 01	
Nombre	Agregar producto	
Referencias		
Autor	Danilo Cerasa, Ignacio Abranchuk, Lara Pollastrini, Santiago Passerini Ignacio Criscenti	
Revisor	Alejandro Sartorio	
Versión	1.0	
Descripción	El usuario podrá agregar un nuevo producto.	
Actores	Usuario	
Pre-condición	El usuario debe tener los permisos correspondientes para poder agregar un nuevo producto.	
Escenario principal	1) El usuario accede a la función "Agregarproducto". 2) El sistema muestra un formulario de ingreso de datos para el nuevo producto. 3) El usuario completa el formulario proporcionando la siguiente información del producto: a) Nombre del producto (campoobligatorio). b) Descripción del producto (campoopcional). c) Categoría del producto (campoobligatorio). d) Cantidad inicial en inventario (campoobligatorio, númeroentero positivo). 4) El usuario confirma la acción de agregar el producto. 5) El sistema validalo los datos ingresados por el usuario. 6) El sistema almacena el nuevo producto en la base de datos del inventario	
Flujos alternativos	1) Cancelar la operación a) El usuario decide cancelar la operación antes de confirmar el formulario. b) El sistema redirige al usuario devuelta a la página principal o a la lista de productos sin realizar ninguna acción. 2) Validación de datos fallida a) El usuario intenta confirmar la acción de agregar el producto. b) El sistema valida los datos ingresados por el usuario y detecta algún error (por ejemplo, campos obligatorios no completados o datos inválidos). c) El sistema muestra mensajes de error junto a los campos correspondientes para indicarlos problemas. d) El usuario corrige los datos según las indicaciones proporcionadas. 3) Error en el procesamiento a) El usuario confirma la acción de agregar el producto. b) El sistema procesa la solicitud pero encuentra un error durante el proceso(por ejemplo,error de conexión a la base de datos). c) El sistema muestra un mensaje de error genérico indicando que la operación no pudo completarse. d) El usuario puede intentar nuevamente más tarde o ponerse en contacto con el soporte técnico para resolver el problema.	
Pos-Condición	El nuevo producto se agrega correctamente al inventario del sistema.	

2. Diseño del sistema

Ejercicio 3: Elabora un diagrama de flujo de datos para la aplicación web del ejercicio 1.

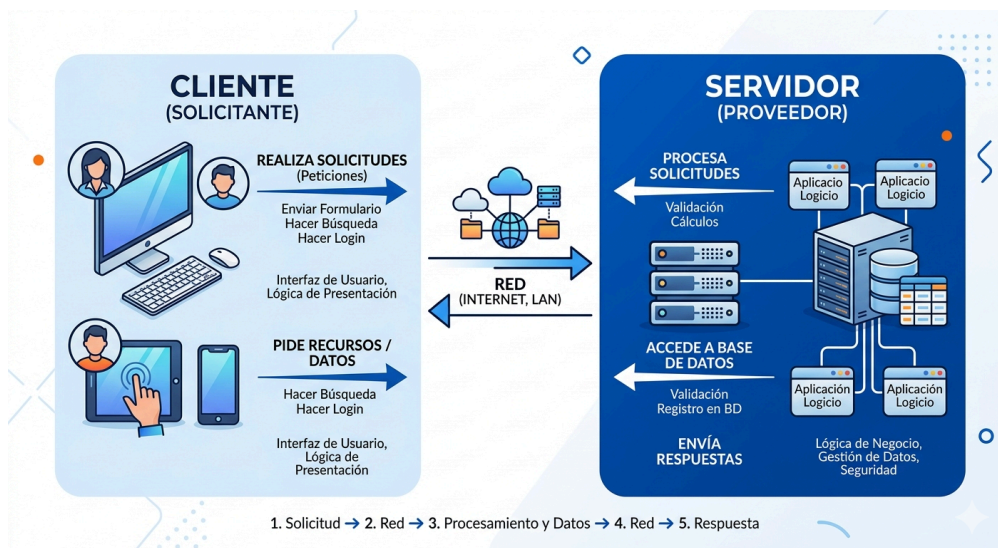


Ejercicio 4: Diseña la interfaz de usuario para la pantalla de "Inicio" de la aplicación web del ejercicio 1.



3. Diseño del programa

Ejercicio 5: Elige una arquitectura adecuada para la aplicación web del ejercicio 1 y justifica tu elección.



Arquitectura cliente-servidor

El Lado del Cliente (Frontend)

Representa la capa de **Presentación**. Es el componente ligero que reside en el dispositivo del usuario. En el sistema de inventario, su función principal es capturar los eventos del teclado y ratón, validar formatos de datos de entrada (como que un precio no sea negativo) y renderizar visualmente la información recibida desde el servidor mediante HTML, CSS y JavaScript.

El Lado del Servidor (Backend)

Representa la capa de **Procesamiento y Lógica**. Es el núcleo donde residen las reglas de negocio. Cuando el cliente solicita "Eliminar un producto", el servidor no solo ejecuta la orden, sino que verifica permisos, comprueba que no existan dependencias (por ejemplo, que el producto no esté en una orden de venta activa) y coordina la escritura final en la base de datos.

El Canal de Intercambio (Request-Response)

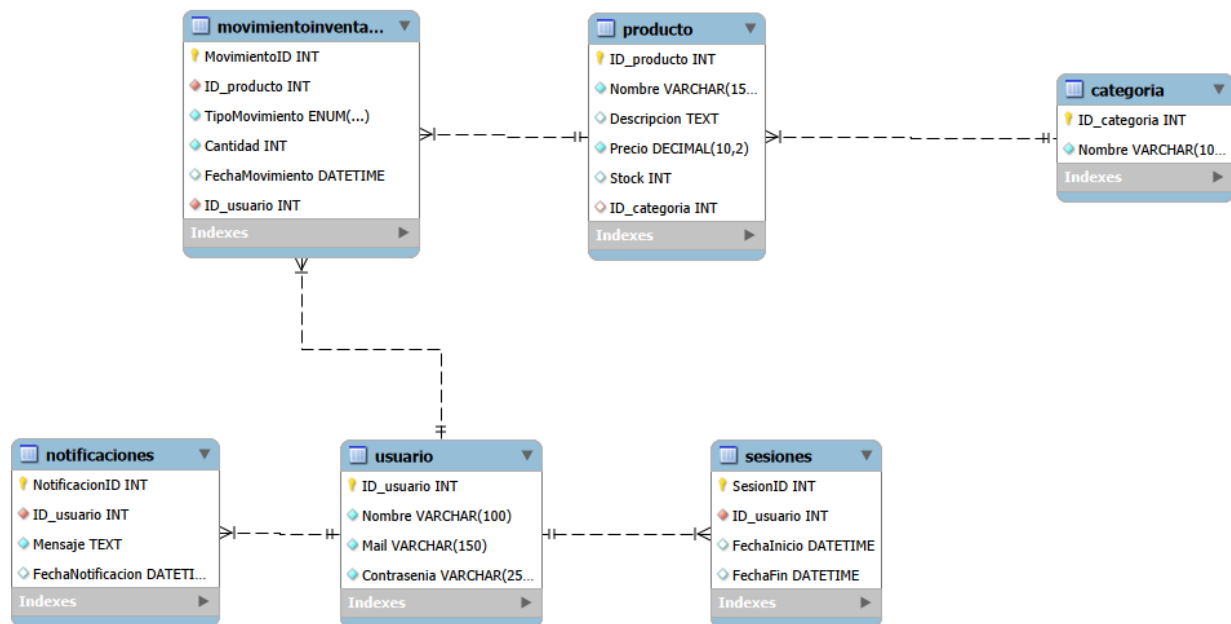
La interacción se basa en un ciclo de **Solicitud-Respuesta**. El cliente inicia la comunicación enviando un paquete de datos (Request) y el servidor, tras procesarlo, devuelve un código de estado (como el **200 OK** o el **404 Not Found**) junto con la información solicitada (Response). Este intercambio ocurre sobre una infraestructura de red segura que garantiza que los datos de stock no sean interceptados.

Justificación de la Elección

Esta arquitectura es la opción predilecta para la gestión de inventarios por tres razones técnicas:

1. **Centralización de Datos:** Evita que haya versiones diferentes del stock; todos los clientes consultan la "única fuente de verdad" en el servidor.
2. **Seguridad de Capas:** Al separar la interfaz de la lógica, los usuarios nunca tienen acceso directo a la base de datos, lo que previene ataques de inyección y manipulación de datos.
3. **Independencia Tecnológica:** Permite actualizar la interfaz visual (el cliente) sin necesidad de modificar cómo se calculan las existencias en el servidor.

Ejercicio 6: Diseña la base de datos para la aplicación web del ejercicio 1.



4. Diseño

Ejercicio 7: Implementa la funcionalidad de "Agregar un nuevo producto" en la aplicación web del ejercicio 1 utilizando el lenguaje de programación de tu preferencia.

index.html

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Inventario Pro - Next Gen</title>
  <link rel="stylesheet" href="styles.css">
  <script src="https://cdn.tailwindcss.com"></script>
  <link
href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@300;500;
700&display=swap" rel="stylesheet">
  
```



```

<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.mi
n.css">
</head>
<body class="app-container">

  <nav class="sidebar">
    <div class="logo">IN.</div>
    <div class="nav-links">
      <button class="nav-btn active"><i class="fas
fa-th-large"></i></button>
      <button class="nav-btn"><i class="fas fa-boxes"></i></button>
      <button class="nav-btn"><i class="fas
fa-history"></i></button>
      <button class="nav-btn"><i class="fas fa-cog"></i></button>
    </div>
    <div class="user-avatar"></div>
  </nav>

  <main class="main-content">
    <header class="top-header">
      <div class="header-text">
        <h1>Sistema de Control <span>v2.0</span></h1>
        <p>Resumen operativo del día de hoy.</p>
      </div>
      <div class="db-status glass">
        <span class="status-dot"></span>
        <span class="status-text">Base de datos: Conectada</span>
      </div>
    </header>

    <section class="hero-stats glass">
      <div class="hero-content">
        <h3>Stock en Movimiento</h3>
        <div class="main-stock"
id="main-stock-display">84,290</div>
        <svg viewBox="0 0 400 100" class="chart-svg">
          <path d="M0,80 Q50,20 100,50 T200,30 T300,70 T400,10"
fill="none" stroke="currentColor" stroke-width="3" />
        </svg>

```

```

        </div>
    </section>

    <section class="action-card">
        <h3>¿Nuevo ingreso al inventario?</h3>
        <p>Registra productos rápidamente en la base de datos.</p>
        <button onclick="toggleModal()" class="add-btn">Añadir
Producto +</button>
    </section>

    <section class="logs-section glass">
        <div class="logs-header">
            <span>ÚLTIMOS_LOGS_SISTEMA</span>
        </div>
        <div id="logs-container" class="logs-body">
            Aún no hay movimientos recientes.
        </div>
    </section>

    <section class="categories-grid">
        <div class="category-card glass">
            <div class="icon-box icon-corralon"><i class="fas
fa-trowel-bricks"></i></div>
            <span class="cat-label">Corralón</span>
            <span class="cat-value text-corralon">60%</span>
        </div>
        <div class="category-card glass">
            <div class="icon-box icon-ferreteria"><i class="fas
fa-wrench"></i></div>
            <span class="cat-label">Ferretería</span>
            <span class="cat-value text-ferreteria">30%</span>
        </div>
        <div class="category-card glass">
            <div class="icon-box icon-otros"><i class="fas
fa-seedling"></i></div>
            <span class="cat-label">Otros</span>
            <span class="cat-value text-otros">10%</span>
        </div>
    </section>
</main>

```

```

<div id="modalAdd" class="modal-overlay">
  <div class="modal-content">
    <h2>Nuevo Producto</h2>
    <form id="productForm">
      <div class="input-group">
        <label>Nombre del Producto</label>
        <input type="text" id="p_nombre" placeholder="Ej:
Cemento Holcim" required>
      </div>
      <div class="input-row">
        <div class="input-group">
          <label>Precio</label>
          <input type="number" id="p_precio"
placeholder="0.00" step="0.01" required>
        </div>
        <div class="input-group">
          <label>Stock Inicial</label>
          <input type="number" id="p_stock" placeholder="0"
required>
        </div>
      </div>
      <div class="modal-actions">
        <button type="button" onclick="toggleModal()"
class="btn-cancel">Cancelar</button>
        <button type="submit"
class="btn-save">Guardar</button>
      </div>
    </form>
  </div>
</div>

<script src="script.js"></script>
</body>
</html>

```

styles.css

```

:root {
  --bg-dark: #0f172a;
  --card-bg: #1e293b;

```

```

--neon-green: #4ade80;
--neon-cyan: #22d3ee;
--neon-purple: #a855f7;
--glass-white: rgba(255, 255, 255, 0.03);
}

body.app-container {
  font-family: 'Space Grotesk', sans-serif;
  background-color: var(--bg-dark);
  color: #e2e8f0;
  margin: 0;
  display: flex;
  height: 100vh;
  overflow: hidden;
}

.sidebar {
  width: 80px;
  background-color: var(--card-bg);
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 2rem 0;
  gap: 2.5rem;
  border-right: 1px solid #334155;
}

.logo { color: var(--neon-green); font-weight: 700; font-style: italic;
font-size: 1.5rem; }
.nav-links { flex: 1; display: flex; flex-direction: column; gap: 2rem; }
.nav-btn { padding: 0.75rem; border-radius: 0.75rem; color: #94a3b8;
transition: all 0.3s; }
.nav-btn.active { background: rgba(74, 222, 128, 0.1); color:
var(--neon-green); }
.user-avatar { width: 40px; height: 40px; border-radius: 50%; background:
linear-gradient(to top right, var(--neon-green), var(--neon-cyan)); }

.main-content {

```

```

    flex: 1;
    padding: 2rem;
    display: grid;
    grid-template-columns: repeat(12, 1fr);
    grid-template-rows: repeat(6, 1fr);
    gap: 1.5rem;
}

.glass { background: var(--glass-white); backdrop-filter: blur(10px);
border: 1px solid rgba(255, 255, 255, 0.1); }

.top-header { grid-column: span 12; display: flex; justify-content:
space-between; align-items: center; }
.header-text h1 { font-size: 1.875rem; font-weight: 700; }
.header-text span { color: var(--neon-green); }
.db-status { padding: 0.5rem 1.5rem; border-radius: 9999px; display: flex;
align-items: center; gap: 0.75rem; }
.status-dot { width: 8px; height: 8px; background: var(--neon-green);
border-radius: 50%; animation: pulse 2s infinite; }

.hero-stats { grid-column: span 8; grid-row: span 3; border-radius: 2rem;
padding: 2rem; position: relative; overflow: hidden; }
.main-stock { font-size: 3.75rem; font-weight: 700; font-style: italic;
margin-bottom: 1.5rem; }
.chart-svg { width: 100%; height: 120px; color: var(--neon-green); filter:
drop-shadow(0 0 8px rgba(74, 222, 128, 0.5)); }

.action-card { grid-column: span 4; grid-row: span 3; background:
var(--neon-green); border-radius: 2rem; padding: 2rem; color:
var(--bg-dark); display: flex; flex-direction: column; justify-content:
space-between; }
.add-btn { background: var(--bg-dark); color: white; padding: 1rem;
border-radius: 1.25rem; font-weight: 700; transition: transform 0.3s; }
.add-btn:hover { transform: scale(1.05); }

```

```

.logs-section { grid-column: span 7; grid-row: span 2; border-radius:
2rem; padding: 1.5rem; }
.logs-header { border-bottom: 1px solid #334155; padding-bottom: 0.5rem;
margin-bottom: 1rem; font-family: monospace; font-size: 0.75rem; }
.logs-body { font-family: monospace; font-size: 0.875rem; color: #94a3b8;
font-style: italic; }

.categories-grid { grid-column: span 5; grid-row: span 2; display: flex;
gap: 1rem; }
.category-card { flex: 1; border-radius: 2rem; padding: 1.5rem; display:
flex; flex-direction: column; align-items: center; justify-content:
center; gap: 0.75rem; transition: all 0.3s; }
.category-card:hover { transform: translateY(-5px); background:
rgba(255,255,255,0.08); }
.icon-box { padding: 1rem; border-radius: 1rem; font-size: 1.25rem; }
.icon-corralon { background: rgba(74, 222, 128, 0.1); color:
var(--neon-green); }
.icon-ferreteria { background: rgba(34, 211, 238, 0.1); color:
var(--neon-cyan); }
.icon-otros { background: rgba(168, 85, 247, 0.1); color:
var(--neon-purple); }
.cat-label { font-size: 0.65rem; font-weight: 700; text-transform:
uppercase; color: #64748b; }
.cat-value { font-size: 1.5rem; font-weight: 700; }

.text-corralon { color: var(--neon-green); }
.text-ferreteria { color: var(--neon-cyan); }
.text-otros { color: var(--neon-purple); }

.modal-overlay { position: fixed; inset: 0; background: rgba(0,0,0,0.8);
display: none; align-items: center; justify-content: center; z-index: 100;
backdrop-filter: blur(5px); }
.modal-content { background: var(--card-bg); padding: 2.5rem;
border-radius: 2rem; width: 450px; border: 1px solid rgba(74, 222, 128,
0.2); }
.input-row { display: grid; grid-template-columns: 1fr 1fr; gap: 1rem; }
.input-group { margin-bottom: 1.25rem; }

```

```
.input-group label { display: block; font-size: 0.7rem; font-weight: 700;
color: #64748b; text-transform: uppercase; margin-bottom: 0.5rem; }
.input-group input { width: 100%; background: rgba(0,0,0,0.2); border: 1px
solid #334155; padding: 0.75rem; border-radius: 0.75rem; color: white; }
.modal-actions { display: flex; gap: 1rem; margin-top: 1rem; }
.btn-cancel { flex: 1; padding: 0.75rem; border: 1px solid #334155;
border-radius: 0.75rem; }
.btn-save { flex: 1; background: var(--neon-green); color: var(--bg-dark);
font-weight: 700; border-radius: 0.75rem; }

@keyframes pulse { 0%, 100% { opacity: 1; } 50% { opacity: 0.4; } }
```

[script.js](#)

```
function toggleModal() {
    const modal = document.getElementById('modalAdd');
    modal.style.display = (modal.style.display === 'flex') ? 'none' :
'flex';
}

document.getElementById('productForm').addEventListener('submit',
function(e) {
    e.preventDefault();

    const nombre = document.getElementById('p_nombre').value;
    const precio = document.getElementById('p_precio').value;
    const stock = document.getElementById('p_stock').value;

    const logsContainer = document.getElementById('logs-container');
    const time = new Date().toLocaleTimeString([], { hour: '2-digit',
minute: '2-digit' });

    const newLog = `
        <div class="log-entry" style="border-left: 4px solid #4ade80;
padding-left: 1rem; background: rgba(255,255,255,0.05); padding: 0.5rem
1rem; margin-bottom: 0.5rem;">
            <span style="color: #4ade80;">[${time}]</span>
            <span style="color: #e2e8f0; margin-left: 10px;">ALTA:
${stock}u de ${nombre.toUpperCase()} (${precio}</span>
        </div>
    `;
    logsContainer.innerHTML += newLog;
});
```

```

        if (logsContainer.innerText.includes("Aún no hay"))
logsContainer.innerHTML = "";
        logsContainer.insertAdjacentHTML('afterbegin', newLog);

        this.reset();
        toggleModal();
    });

```

Ejercicio 8: Implementa la lógica de negocio para la funcionalidad de "Agregar un nuevo producto" en la aplicación web del ejercicio 1.

La lógica de negocio para la implementación de la funcionalidad **Agregar un nuevo producto** en el sistema **IN**, comienza con un control de acceso riguroso a través de la pantalla de inicio de sesión. En esta instancia, el usuario debe ingresar su correo electrónico y contraseña, los cuales son validados por el backend consultando la tabla **Usuario**. Si las credenciales son correctas, el sistema registra el evento en la tabla **Sesiones** almacenando el **ID_usuario** y la marca temporal de inicio, para luego redirigir al operador hacia el dashboard principal.

Una vez dentro de la interfaz operativa, el usuario navega a través del menú lateral hasta la sección de gestión de inventario. Al interactuar con el botón de **Añadir Producto**, se despliega un formulario modal que captura los datos esenciales del artículo, tales como su nombre, descripción, precio y stock inicial. En este punto, el backend ejecuta un proceso de validación crítica para garantizar que los campos obligatorios estén completos y que el formato de los datos sea el adecuado; por ejemplo, se verifica que el precio sea un valor decimal positivo y que la categoría seleccionada exista previamente en la tabla **Categoria** para mantener la integridad referencial.

Si la validación resulta exitosa, el sistema procede a la persistencia de los datos realizando una inserción en la tabla **Producto**, lo que genera un identificador único de forma automática. Inmediatamente después, la lógica de negocio exige registrar esta entrada en la tabla **MovimientoInventario**, vinculando el producto con el usuario responsable y estableciendo el tipo de movimiento como una entrada para asegurar la trazabilidad total del stock.

Finalmente, el sistema notifica al usuario el éxito de la operación mediante un mensaje en pantalla y actualiza el flujo de **Logs de Sistema** en tiempo real para reflejar el nuevo

ingreso. Toda esta información impacta directamente en las métricas de las categorías de **Corralón, Ferretería y Otros**, cuyos porcentajes se recalculan automáticamente. Además, el usuario puede dirigirse a la sección de gráficos para visualizar o descargar reportes detallados sobre el estado actualizado del inventario y los movimientos recientemente ejecutados.

5. Pruebas

Ejercicio 9: Define un conjunto de pruebas unitarias para la funcionalidad de "Agregar un nuevo producto" en la aplicación web del ejercicio 1.

Para el **Ejercicio 9**, he adaptado las pruebas unitarias a la estructura técnica de tu proyecto **IN.**, considerando tanto las restricciones de tu base de datos `tp_inventario` como la lógica de interfaz que venimos desarrollando.

La primera prueba unitaria se centra en la inserción exitosa de un producto con datos válidos en el sistema. El proceso comienza al configurar un objeto con un nombre descriptivo, una descripción técnica, un precio decimal positivo y un `ID_categoria` que exista en la tabla maestra. Al ejecutar la función de guardado, el sistema debe procesar la solicitud, generar el nuevo registro en la tabla `Producto` y disparar un movimiento de entrada en `MovimientoInventario`. El resultado esperado es una confirmación de éxito y la aparición automática de un nuevo log de sistema en el dashboard principal.

La segunda prueba evalúa la robustez del sistema ante la ausencia de datos obligatorios, específicamente el nombre del producto. En este escenario, se intenta llamar a la función de guardado enviando el campo de nombre vacío o nulo, manteniendo el resto de los parámetros correctos. El sistema debe interceptar esta omisión mediante una validación de seguridad antes de intentar cualquier operación en la base de datos. El resultado esperado es que la función rechace la solicitud y devuelva un mensaje de error notificando que el nombre del producto es un campo obligatorio para proceder con el registro.

La tercera prueba unitaria verifica la integridad de los datos numéricos, enfocándose en el precio del producto. Para este test, se ingresa un valor negativo en el campo de precio mientras se completan correctamente los demás datos como el stock y la categoría. La lógica de negocio debe detectar que un valor menor a cero es inválido para un activo del inventario. El resultado esperado es que el sistema bloquee la inserción y emita una alerta indicando que el precio del producto debe ser un valor mayor o igual a cero, protegiendo así la coherencia financiera de la base de datos.

Ejercicio 10: Ejecuta pruebas de integración para la funcionalidad de "Agregar un nuevo producto" en la aplicación web del ejercicio 1.

La primera prueba de integración se enfoca en validar la persistencia real en el motor de base de datos MySQL. El proceso se inicia ejecutando la función de alta desde la aplicación, tras lo cual se realiza una consulta directa mediante un SELECT en la tabla Producto de la base de datos tp_inventario. El objetivo es confirmar que el registro no solo existe, sino que sus campos de nombre, precio y stock coinciden exactamente con los valores capturados en el formulario modal del frontend. El resultado esperado es una sincronización perfecta entre la entrada de datos del usuario y el almacenamiento físico en el servidor.

La segunda prueba de integración verifica la comunicación entre el cliente y el servidor mediante el protocolo HTTP. Para este test, se simula el envío de una solicitud POST al backend con el cuerpo del nuevo producto en formato JSON. Se captura la respuesta del servidor para validar que retorne un código de estado 200 OK, lo que confirma que la lógica de negocio procesó la solicitud sin errores internos. El resultado esperado es que el backend devuelva una confirmación técnica que permita a la interfaz web disparar la notificación de éxito y actualizar el componente de "Logs de Sistema" que se visualiza en el panel principal.

La tercera prueba de integración comprueba la actualización dinámica de la interfaz de usuario tras una inserción exitosa. Una vez que el producto ha sido guardado, el flujo de la prueba navega hacia la sección de gestión de inventario para inspeccionar la lista de existencias. Se verifica visualmente y a través del DOM que el nuevo artículo (como por ejemplo "Mezcladora" o "Cemento Holcim") aparezca listado con su información correspondiente. El resultado esperado es que el nuevo producto sea plenamente visible para el operador, garantizando que la vista se refresca correctamente con la información más reciente de la base de datos.

6. Despliegue del programa

Ejercicio 11: Definir un plan de despliegue para la aplicación web del ejercicio 1.

Plan de Despliegue: Sistema de Control de Inventario IN.

La fase inicial se centra en la preparación del entorno de producción, donde se designa un servidor robusto para alojar la aplicación. En este servidor se debe instalar y configurar el stack tecnológico necesario, que incluye el servidor web (como Nginx), el entorno de ejecución para la lógica de negocio y el sistema de gestión de bases de datos MySQL, asegurando que la base de datos `tp_inventario` esté correctamente inicializada con todas las tablas y restricciones de integridad.

Una vez listo el entorno, se procede al empaquetado de la aplicación, compilando todos los archivos de diseño en `styles.css`, la lógica de interacción en `script.js` y la estructura de `index.html` en un paquete distribuible. Este paquete debe incluir todos los activos visuales, como los iconos de categorías y la tipografía Space Grotesk, garantizando que no falte ningún recurso crítico para la interfaz.

Antes del lanzamiento final, se configura un entorno de preproducción que actúa como una réplica exacta del sistema real. En esta etapa se realizan pruebas exhaustivas para verificar que el formulario modal de "Añadir Producto" y las tarjetas de estadísticas de **Corralón, Ferretería y Otros** funcionen sin errores en un entorno controlado. Tras validar la estabilidad, se coordina el despliegue en producción en un horario de bajo tráfico para minimizar el impacto en los usuarios, transfiriendo y configurando los archivos en el directorio final del servidor.

Posterior al despliegue, se ejecuta una verificación rigurosa en el entorno de producción para confirmar que todas las funciones, especialmente la inserción de nuevos productos y la actualización de los logs, operen según lo esperado. Finalmente, tras obtener la aprobación del equipo de calidad, se comunica a los interesados y usuarios finales que la versión **v2.0** del sistema está disponible, proporcionando soporte para asegurar una transición fluida a la nueva plataforma de control.

Ejercicio 12: Despliega la aplicación web del ejercicio 1 en un servidor de producción

La puesta en marcha comienza con la intervención del **equipo de desarrollo**, el cual se encarga de consolidar la versión final del código, asegurando que el archivo `styles.css` esté optimizado y que el `script.js` apunte correctamente a las rutas del servidor. Paralelamente, el **equipo de operaciones** toma el control de la infraestructura para configurar el servidor de producción, estableciendo las reglas de seguridad necesarias y preparando el motor MySQL para gestionar la base de datos `tp_inventario` en un entorno de alta disponibilidad.

Un paso crítico en esta fase es la configuración del **hosting** y la vinculación del nombre de dominio oficial de la aplicación. Esto permite que el sistema de control de inventario deje de ser un proyecto local y se convierta en una plataforma profesional accesible desde internet. Una vez que el dominio está activo y apuntando al servidor, el equipo de operaciones levanta los servicios web, permitiendo que la interfaz con estética *Glassmorphism* sea renderizada correctamente en cualquier navegador.

Tras la activación, el **equipo de calidad (QA)** inicia una ronda de pruebas de verificación en el entorno real, ejecutando tareas como el registro de un producto de prueba para confirmar que la integración entre el frontend y la base de datos sea impecable bajo condiciones de tráfico real. Finalmente, una vez validada la estabilidad, se presenta el sistema al **cliente y a los usuarios finales**, quienes realizan la aprobación definitiva y comienzan a utilizar la herramienta para la gestión operativa diaria, logrando así un despliegue exitoso y funcional.

7. Mantenimiento

Ejercicio 13: Definir un plan de mantenimiento para la aplicación web del ejercicio 1.

El primer pilar del plan consiste en un **monitoreo continuo**, donde se establecen sistemas automatizados que supervisan el rendimiento y la disponibilidad de la aplicación en tiempo real. Esto implica revisar periódicamente los logs del servidor para detectar cualquier intento de acceso fallido o errores en las consultas a la base de datos `tp_inventario`. Al mantener una vigilancia constante sobre las alertas del sistema, el equipo técnico puede intervenir antes de que un problema afecte la experiencia del usuario o la integridad del stock.

En cuanto a la **seguridad y optimización**, el plan exige mantener actualizadas todas las dependencias y librerías, como Tailwind CSS y Font Awesome, para mitigar posibles vulnerabilidades. Asimismo, se realizan tareas de optimización del rendimiento

enfocadas en la eficiencia de las consultas SQL y en la velocidad de respuesta del dashboard. Esto garantiza que, a medida que la cantidad de productos en las categorías de **Corralón, Ferretería y Otros** aumente, la interfaz siga cargando de manera fluida y el consumo de recursos en el servidor se mantenga bajo control.

Finalmente, el mantenimiento incluye un ciclo de **actualizaciones y resguardo de información**. Se establece un proceso para recopilar sugerencias de los usuarios, permitiendo que el sistema evolucione con nuevas funciones que mejoren la gestión del inventario. Para proteger toda la información operativa, se implementa un programa de **copias de seguridad automatizadas**, asegurando que ante cualquier fallo técnico o pérdida de datos, la base de datos pueda ser restaurada a su estado más reciente sin interrumpir la continuidad del negocio del cliente.

Ejercicio 14: Implementa una corrección de errores para un problema detectado en la aplicación web del ejercicio 1.

La gestión de incidencias comienza con la **identificación y análisis del problema**, donde el equipo de soporte reporta, por ejemplo, una inconsistencia en el cálculo automático de los porcentajes de las tarjetas de **Corralón y Ferretería**. Ante esto, el equipo de desarrollo examina el código en script.js y la base de datos tp_inventario para comprender la raíz del error, que podría deberse a una falta de actualización en los triggers de MySQL tras una eliminación masiva de registros.

Una vez detectada la causa, se procede a la **corrección del error** mediante la modificación del script de lógica de negocio, asegurando que las operaciones aritméticas contemplen todos los estados posibles del stock. Antes de cualquier lanzamiento, se ejecutan **pruebas de verificación** exhaustivas en un entorno de desarrollo para garantizar que la solución no solo resuelva el fallo detectado, sino que tampoco genere efectos colaterales en otras funciones críticas, como el formulario de alta de productos.

Finalmente, la corrección se traslada al **entorno de producción** mediante un despliegue controlado. Tras confirmar que el sistema vuelve a reflejar los valores exactos en el dashboard, se obtiene la **aprobación definitiva** por parte del equipo de calidad. El proceso concluye con la **comunicación y documentación** del cambio, informando a los operadores del sistema sobre la mejora realizada y registrando la solución en el historial técnico, dejando la plataforma robustecida y preparada para enfrentar nuevos retos operativos y futuras escalabilidades.

8. Nos preparamos para nuevos retos

Ejercicio 15: Arme un equipo de trabajo y defina los roles para realizar los ejercicios anteriores para un futuro dominio de aplicación relacionado con inteligencia artificial generativa.

La dirección estratégica recae en el **Gerente de Proyecto**, quien actúa como el nexo entre los objetivos del cliente y la ejecución técnica. Su función principal es coordinar los esfuerzos de todas las áreas, asegurando que el desarrollo de la solución de IA avance bajo los cronogramas previstos y gestionando los recursos necesarios para que el equipo pueda iterar rápidamente en un entorno tan dinámico como el de la IA generativa.

En el núcleo técnico, el **Desarrollador** se encarga de construir la arquitectura de la aplicación y las interfaces que permitirán al usuario interactuar con la IA, mientras que el **Científico de Datos** asume la responsabilidad de seleccionar, entrenar y ajustar los modelos generativos. Este último se enfoca en optimizar la precisión y la coherencia de las respuestas del modelo, trabajando en el refinamiento de los conjuntos de datos para evitar sesgos y mejorar la calidad de los resultados generados.

Para sostener estas operaciones de alto cómputo, el **Ingeniero de Infraestructura** diseña y mantiene la arquitectura en la nube, gestionando el despliegue de los modelos en servidores escalables que soporten la carga de procesamiento necesaria para la IA. Por su parte, el **Analista de Calidad o QA** implementa protocolos de prueba específicos para inteligencia artificial, evaluando no solo el código, sino también la veracidad y seguridad de los contenidos producidos por el modelo para garantizar un producto confiable.

Finalmente, el equipo se completa con la participación activa de los **Stakeholders**, quienes definen los requisitos de negocio y proporcionan el feedback necesario para alinear la tecnología con las necesidades del mercado. Este esquema de trabajo colaborativo asegura que el dominio de aplicación de IA generativa no solo sea técnicamente avanzado, sino también una herramienta útil, segura y alineada con las expectativas reales de los usuarios finales.